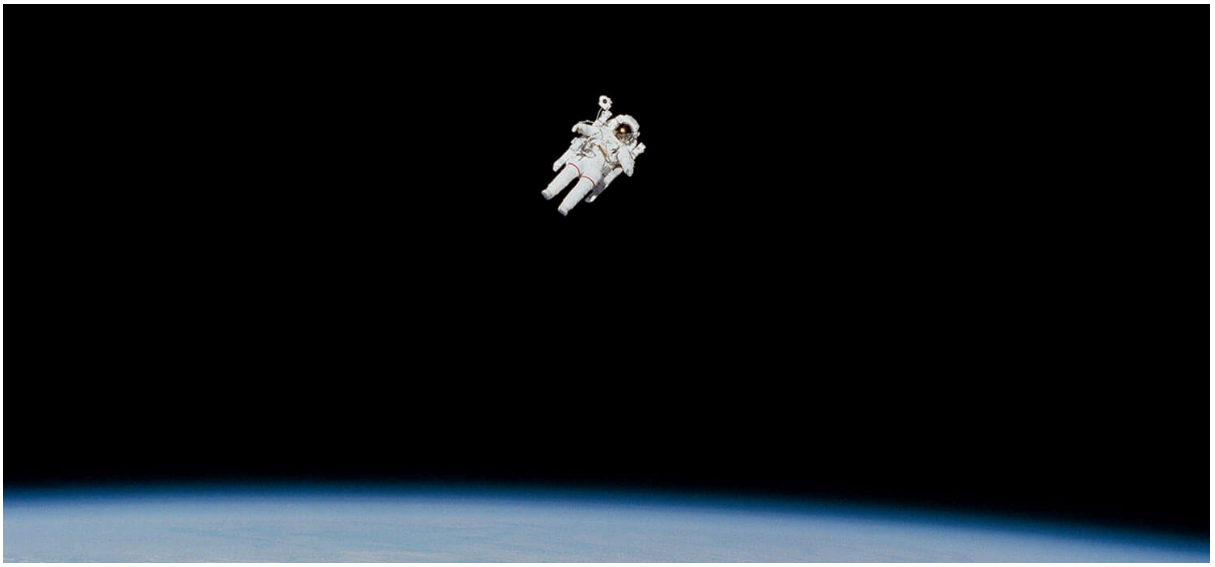


- JavaScript → Add functionality to our web page
- REPL → Read, Evaluate, Print, Loop.
- Variable
- Data types in JS → Number, boolean, String, Undefined, null.
- Numbers in JS.
- Operators in JS. (+ , - , * , / , % , **)
- NaN.
- Operator Precedence
  - ( )
  - **
  - * , / , %
  - + , -
- Assignment operator
  - age = age + 1
  - age += 1.
- Unary operator [age++]
  - Pre-increment → Update then use → ++age
  - post-increment. → use then update. → age++

- variable identifier rules.
- camelCase, snake_case, PascalCase
- Boolean
- String
- String Indices.
- ‘name’.length.
- Concatenation
- null, undefined.
- Console.log()
- Template literals
- Comparison operators. (>, >=, <, <=, ==, !=, ===)
- Type of variable
- Comparison for non-numbers.
- Conditional Statement. (if-else) (nested if-else) (Switch)
  - (if)
  - (else if)
  - (else statement)
  - (Nested if else)
- Logical operators (&& , ||)
- Truthy and falsy values.
- Switch statement.
  - Break
  - default
- Alert and prompt.
- delta 2
- String. Method ()
  - **trim()**

    - **string is immutable**
    - **toUpperCase()**
    - **toLowerCase()**
- String. method (arg)
    - **String. indexOf("")**
- Method Chaining.
    - **String. toUpperCase().trim()**
- String. Slice (Start, end).
- **String. replace (oldvalue, new value)**
- **String. repeat (3)**
- Array →An **array** is a data structure that stores **multiple values** in a **single variable**, arranged in an **ordered list** and accessed using an **index**.
- Array[0].length.
- Array. push ()
- array. pop ()
- array. unshift ()
- array. shift ()
- array.at() → this method takes an integer value and returns the item at that index.
  array.at(1) → this will give the 2nd item of the array
  array.at(-1) → this will give the last item of the array
- array. index of ("yellow")
- array. includes ("yellow")
- array1. Concat (array2).
- array. reverse ().
- array. Slice (start, end index).
    - Slice → copies a portion of an array.
    - splice → removes/replaces/ add elements → add the new element at the same position from where we remove something

- array. Splice (0,1, "black", "grey")
- array. Sort ()
- ▼ Difference betN array.some VS array.find
  - `.some()` → **returns a boolean**
    `.find()` → **returns the element itself**
  - ```
    const users = [
    { id: 1, active: false },
    { id: 2, active: true },
    ];
    users.some(user ⇒ user.active);
    // true
    ```
  - ```
    const users = [
    { id: 1, active: false },
    { id: 2, active: true },
    ];
    users.find(user ⇒ user.active);
    // { id: 2, active: true }
    ```
- array references
- array Mutable
- Const array = [ ]
- Nested array.
- Loops
  - for loop.
  - Dry Run
  - infinite loop
  - Nested for loop
  - while loop
  - break Keyword.
  - loops with array

- loops with nested array.
- for of loop.
- Nested for of loop.

delta3

- object literals.
  - Property: (key, value) Pairs.
  - Const item = {
    Price: 109.99,
    discount: 50,
    Colours : ["red", "Pink"]
    };
- Get values
  - Object["Key"]
  - Object.Key
  - Add/update values.
    - Object.Key = "New value"
    - Object.Newkey = "New value"
  - delete Object.key
- Object to objects Nesting.
- Array of objects.
- Object[i].Key →when we have a array of objects then to access a key of an object which object is in the mentioned index value.
- Math Objects.
  - Properties. →
    Math.PI
    Math.E
  - Methods. →
    - Math.abs(n) → To get absolute value
    - Math.pow(a,b) → To get Powered value aba to the b-th powerab

    - Math.floor(n) → To get round off value in decreasing form
    - Math.ceil(n) → To get round off value in increasing form
    - Math.random() →To get random values.
- Functions in JS.
  Syntax →

```
function funcName() {
// do something
}
```

- Functions with Arguments.

```
function funcName (arg1, arg2,.....) {
// do something
}
```

- Return Keyword.
- **Scope**
    - function
    - Block
    - Lexical
- Function expression

```
Const variable = function (arg1, arg2) {
// do or return something
};
```

  Variable (a,b);

- Higher Order function → Take multiple func as argument or return a function.
- Methods → Action that can be performed on an object.
- 'This' Keyword → 'this' Keyword refers to an object that is executing the current piece of code.
- Try and Catch.
- Miscellaneous topics:
    - arrow function `const func1 = (arg1, arg2) => { function defination }`.
    - Set Time out `setTimeout(function, timeout)`.

- `setInterval(function, timeout)` . `clearInterval(id);`
- 'This' Keyword with arrow functions → here 'this' Keyword refers to the parent of that object who is executing the current piece of code.

Delta 4

- array Methods.
  - foreach `arr.foreach (some function defination or name);`
    when no need to return anything and no need to create a new array use it.
  - map `arr.map (some function defination or name)` .
    when we need to return something and need a new array use it.
  - filter. `arr.filter (some function defination or name)`
  - every → `arr. every (Some function defination or name)`
  - Some → `arr. some (Some function defination or name)`
- Reduce → `arr. reduce (reducer function with 2 variable for (accumulator, element));`
- Default Parameter
  function func (a, b=2) {
  // do something
  }
- Spread → Spread **copies values** from an array or object into another array or object.
  Spread with array
  Spread with object literals.

  ▼ Example

  const a = [1, 2];
  const b = [...a, 3];
  [1, 2, 3]

▼ Rest

Rest parameter (`...` **)** collects **multiple values into an array**.

- function sum(...numbers) {
  return numbers;
  }
  sum(1, 2, 3, 4);
    - output = [1, 2, 3, 4]
- function add(a, b, ...rest) {
  console.log(rest);
  }
  add(1, 2, 3, 4, 5);
    - output = [3, 4, 5]
- Rest must be the **last parameter**
    - ❌ function test(...a, b) {}
      ✅ function test(a, b, ...rest) {}

- Destructuring .
- DOM (document object model)
    - **DOM (Document Object Model)** is a **tree-like structure** created by the browser from your HTML. JavaScript uses the DOM to **read, change, add, or remove** HTML elements.
        - html → Hello
          js → document.getElementById("text").textContent = "Hi";
          result → The text changes from **Hello → Hi**
    - Document tree → The **Document Tree** shows how HTML elements are **nested like a family tree**.
        - html =
          <html>
          <body>
          Hello
          </body>
          </html>

          tree view =
          document
          └── html

└── body
    └── p

  - node = document tree t thaka every individual element ba document k Node buli koi
  - document object. = The `document` **object** is the **entry point** to the DOM. It represents the **entire web page , ay document object tur use kri a ami javascript t html ru css k manipulate kribo pru . means document object** lets JavaScript access and control HTML.
  - Console.dir()

- getElementById
  document.getElementById("Id Name")
- getElementByClassName
  document.getElementsByClassName("Class Name")
- getElementByTagName
  document.getElementsByTagName("Tag Name")
- Query Selectors.
  - document.querySelector('p')
  - document.querySelector("#myId")
  - document.querySelector(".my class")
  - document.querySelectorAll("p")
- InnerText →
  Hello Hidden World
  innerText = Hello World
- textContent
  Hello Hidden World
  textContent = Hello Hidden World
- innerHTML
  Hello World
  innerHTML = Hello World
- Obj.getAttribute(attr)
  Obj.setAttribute(attr, val)

- Obj.style
  let heading = document.querySelector('h1')
  heading.style.color = 'purple'
- Obj.classList
  - classList.add() → To add new class
  - classList.remove() → To remove new class
  - classList.contains() → To check if class exist
  - classList.toggle() → To toggle both add and remove.
  - Eg -
    let heading = document.querySelector('h1');
    heading.classList.add('abc');
- Navigation
  - Parent Element
  - Children
  - Previous Element Sibling / next Element Sibling.
  - Eg -
    let h1 = document.querySelector ('h1');
    h1.parentElement
    output = ...
- document.createElement ('p') → Adding element
  - appendChild (element)
  - append (element)
  - prepend (element)
  - insertAdjacentElement (where, element)
  - 'beforebegin'
  - 'afterbegin'
  - 'beforeend'
  - 'afterend'
- Removing element

    - removeChild (element)
    - remove (element)

- Dom element events.
    - onclick
    - onMouseEnter
- Event listener
    - addEventListener.
      element.addEventListener(event, callback)

      btn.addEventListener(“click” , ()⇒{
      console.log(“ button clicked “)
      }
- this in event listener. → when ‘this’ is used in a callback of event handler of something , it refers to that something .
- Console.dir()
- Keyboared events
- Change event
- input event.
- Event Bubbling. → **Event bubbling** means when you click an element, the event first runs on that element and then **moves up** to its parent elements.
  { event.stopPropagation() → to stop event bubbling we use this}
- Event Delegation. → **Event delegation** means putting **one event listener on a parent** to handle events for **many child elements**.
- Call stack → Functions go **in**, run, and then come **out** — **Last In, First Out (LIFO)**.
- Breakpoints. → A **breakpoint** is a **pause point** in your JavaScript code where execution stops so you can inspect what’s happening.
- Js is Single Threaded → JavaScript can do **only one thing at a time**. It runs code **line by line**, not multiple things at the exact same time.

if a code takes time to run then everything else will wait to complete , means it blocks the other code to run, there comes ASYNC it helps to run code without blocking the execution.

- Callback Hell → **Callback hell** happens when you put **callbacks inside callbacks inside callbacks**, making code **hard to read and maintain**.
- Promise.
  A Promise is a container for a future value.
  You get the value using `.then()` for success and `.catch()` for failure.
  - (resolve, reject)
  - (async function)
  - (.then(), .catch())
- Async function → makes a function return a Promise
- Await Keyword.
  → waits for a Promise to finish
  await pauses the code until the Promise finishes.
- API → application programming interface
  → a way for two software systems to talk to each other.
  a pipeline through which your app sends and receives data from a server.
  - JSON → JAVASCRIPT OBJECT NOTATION
    A **lightweight, easy-to-read** format for sending data between server and client. simple, clean data format that looks like JS objects.
  - XML → An **older, heavier, tag-based** format like HTML used to structure data.
  - JSON.parse(data). → `JSON.parse()` converts a JSON string → normal JavaScript object
  - JSON.stringify(data) → `JSON.stringify()` converts a **J**avaScript object → JSON string.
  - AJAX → Asynchronous JavaScript And XML
  - AJAJ → Asynchronous JavaScript And JSON
    AJAX / AJAJ is an asynchronous process of API request and response without reloading the page.
    here
    Asynchronous → request runs in background

API request → send data to server
Response → get data back
No page reload → UI stays smooth

- HTTP verbs → **HTTP verbs** are **actions** that tell the server **what you want to do** with data.
    - Common HTTP Verbs (Simple)
      GET → get data
      POST → send new data
      PUT → update all data
      PATCH → update part of data
      DELETE → remove data
- Status Code →
    - 1xx – Informational
      Rarely used
      `100 Continue` → server says "go on"
    - 2xx – Success
        - Request was successful
        - **Common ones:**
            - `200 OK` → success
            - `201 Created` → resource created
            - `204 No Content` → success but no data
    - **3xx – Redirection**
        - Browser needs to go somewhere else
        - **Common ones:**
            - `301 Moved Permanently` → URL changed
            - `302 Found` → temporary redirect
    - **4xx – Client Errors**
        - Something wrong with the request
        - **Common ones:**
            - `400 Bad Request` → invalid request

        - `401 Unauthorized` → need login
        - `403 Forbidden` → no permission
        - `404 Not Found` → resource doesn't exist
        - `409 Conflict` → duplicate or conflict
    - **5xx – Server Errors**
        - Server failed to process request
        - **Common ones:**
            - `500 Internal Server Error` → generic error
            - `502 Bad Gateway` → server got invalid response
            - `503 Service Unavailable` → server overloaded/down
- Query String in URL. → A **query string** is the part of a URL that **sends data to the server**, usually after `?`.
    - https://example.com/search?query=javascript&sort=latest
      ? → starts the query string
      query=javascript → first key-value
      & → separates multiple key-values
      sort=latest → second key-value
- HTTP headers. →
    - **Headers** are **metadata sent with HTTP requests or responses**.
      They tell the server/client **how to handle the request or response**.
    - Types of Headers
        - Request Headers (sent by client)
            - `Content-Type` → type of data being sent (`application/json`)
            - `Authorization` → token for authentication (`Bearer <token>`)
            - `Accept` → types of data client can accept (`application/json`)
            - `Cookie` → send cookies to server
        - Response Headers (sent by server)
            - `Content-Type` → type of data returned (`application/json`)
            - `Set-Cookie` → set cookies in browser

        - `Cache-Control` → caching rules
        - `Access-Control-Allow-Origin` → CORS
- Fetch(url)
- Axios